

Sistema di Logistica Droni Resiliente con AI Agentica

Lorenzo Amorosa

Edoardo Buttazzi

Gregorio Cavulla

Indice

1	Introduzione e Definizione del Servizio	2
1.1	Dominio Operativo	2
1.2	Stakeholder e Utenti Target	2
1.3	Workload e Threat Model	2
2	Architettura del Sistema e Osservabilità	4
2.1	Design a Microservizi	4
2.2	Simulazione del Mondo Fisico (Digital Twin)	4
2.3	Strategia di Osservabilità	4
3	Agentic AI e Design MCP	6
3.1	Il Limite dell'Automazione Deterministica	6
3.2	Architettura MCP	6
3.3	Health Agent e Logistic Agent	6
3.4	Orchestrazione Dinamica	6
4	Operational Safety e Agent Guardrails	7
4.1	Least Privilege e Protezione Anti-Loop	7
4.2	Idempotenza delle Azioni	7
4.3	Human-In-The-Loop ed Escalation	7
5	Reliability e Scalabilità	9
5.1	Resilienza di Rete	9
5.2	Graceful Degradation del Server Centrale	9
5.3	Scalabilità Dinamica e Safety Limits	9
6	Chaos Engineering e Test di Fallimento	10
6.1	Metodologia	10
6.2	Esperimenti e Risultati	10
6.3	Lezioni Apprese	11
7	Analisi Economica: SLO, Costi e ROA	12
7.1	SLO e Costo del Downtime	12
7.2	Return on Agent (ROA): OPEX e CAPEX	12
7.3	Trade-off Costo/Affidabilità e Automazione/Sicurezza	13
8	Conclusioni e Sviluppi Futuri	14

1 Introduzione e Definizione del Servizio

1.1 Dominio Operativo

Il progetto si colloca all'intersezione tra *Transportation* e *Industrial / IoT Monitoring* e implementa un'architettura logistica autonoma per una flotta di droni adibita alla consegna urbana di payload critici (forniture mediche, organi, componenti industriali ad alta priorità). In un servizio *high-stakes* di questo tipo, efficienza e affidabilità sono requisiti fondamentali.

Il sistema opera in un'area metropolitana di raggio 5 km attorno a un HUB centrale in $[0.0, 0.0]$. Gli ordini hanno peso 0.5–5.0 kg e priorità *low/normal/high*. I droni sono modellati come agenti fisici con quattro stati operativi (IDLE, IN_DELIVERY, RETURNING, MAINTENANCE): la velocità in consegna varia tra 50 e 80 km/h in funzione del carico, mentre in rientro a vuoto raggiunge i 100 km/h. Ogni missione accumula usura (0–100%) e il superamento di soglie critiche richiede manutenzione preventiva per evitare schianti.

L'elevata variabilità delle condizioni (es. pacco da 4.5 kg *high priority* con droni a bassa batteria o alta usura) rende inefficace l'automazione deterministica. Il sistema delega quindi il triage a un layer *Agentic AI* che bilancia in tempo reale rischio operativo, urgenza e salute della flotta.

1.2 Stakeholder e Utenti Target

Stakeholder. (i) *Operatori Human-in-the-Loop*, che validano o respingono via gateway le azioni ad alto impatto degli agenti e monitorano la dashboard real-time; (ii) *Ingegneri DevOps/SRE*, responsabili della salute di nodi/Pod Kubernetes, broker Mosquitto e database InfluxDB, che intervengono in caso di escalation infrastrutturale; (iii) *Business Owner e Controller Finanziari*, che monitorano il bilanciamento CAPEX/OPEX e il *cost ceiling* legato al consumo di token, con KPI di riferimento il *Return on Agent* (ROA), che misura il risparmio derivante dalla manutenzione predittiva rispetto al costo vivo delle API LLM.

Utenti target. (i) *Entità mittenti* (hub medici, e-commerce critici, hub industriali) che immettono ordini in modo asincrono e richiedono il rispetto degli SLO, soprattutto per gli ordini *high priority* anche durante *load spikes*; (ii) *Destinatari finali* (laboratori di analisi, linee di produzione *Just-In-Time*) per i quali downtime e fallimenti catastrofici (schianto del drone in volo per esaurimento batteria) non sono negoziabili.

1.3 Workload e Threat Model

L'architettura è *event-driven* con carico ibrido:

- **Telemetria IoT costante:** ogni Pod `drone_simulator` pubblica un messaggio JSON su `telemetry/drones` ogni 2s; il `central_server` aggiorna lo stato in RAM e persiste su InfluxDB.
- **Eventi di business asincroni (spiky):** il `client_simulator.py` immette ordini su `business/ordini/nuovi`, con picchi anomali negli scenari di stress.
- **Polling agentic read-heavy:** l'orchestratore interroga InfluxDB (time-window di 5–60 min) e l'API Kubernetes a ogni risveglio, richiedendo latenze minime per non violare gli SLO di reattività.

I principali scenari di guasto baseline sono: caduta del broker MQTT (mitigata da rischedulazione K8s + *exponential backoff* di `paho-mqtt`), indisponibilità di InfluxDB (mitigata da *graceful degradation* del server centrale), picchi di carico anomali (gestiti dallo scaling agentico) e allucinazioni/loop dell'LLM (contenuti dal layer MCP e dai contatori di iterazione, Capitolo 4).

2 Architettura del Sistema e Osservabilità

L'architettura è basata su Kind (Kubernetes IN Docker), che avvia cluster Kubernetes locali in container Docker, replicando scheduler, networking e controller di un cluster reale in un ambiente facilmente ricreabile per validazione e test. Kubernetes orchestra i container organizzandoli in Pod, mantiene lo stato desiderato tramite **Deployment** e **ReplicaSet**, espone i servizi con **Service** e fornisce funzionalità native di autoscaling e self-healing. Docker fornisce la base di containerizzazione che confeziona applicazioni e dipendenze in immagini portabili, garantendo isolamento, avvii rapidi e coerenza tra ambienti.

2.1 Design a Microservizi

Il sistema separa responsabilità e domini di failure in tre blocchi indipendenti, ciascuno in Pod distinti con **Service** per la service discovery via DNS interno (**influxdb-service**, **mosquitto-service**):

- **Data Layer** (InfluxDB): persistenza di metriche e serie temporali, stateful con PVC.
- **Message Broker** (Mosquitto/MQTT): canale asincrono che disaccoppia produttori e consumatori, permettendo riavvii e scaling del layer logico senza spezzare l'ingest.
- **Logic Layer** (**central_server**, MCP, agenti): elaborazione e API di business.

Gli script Python sono eseguiti come processo principale del container tramite il **command** del Deployment; lo scheduler decide il nodo, il campo **replicas** distribuisce le istanze su nodi diversi mantenendo invariata l'interfaccia grazie ai **Service**.

2.2 Simulazione del Mondo Fisico (Digital Twin)

I digital twin di droni e client sono generatori controllati di telemetria e domanda. Il twin del drone (**drone_simulator.py**) mantiene stato interno (posizione, batteria, usura, stato operativo), pubblica telemetria su **telemetry/drones** e ascolta comandi su **comandi/<DRONE.ID>**. Il twin del client (**client_simulator.py**) genera carico business pubblicando ordini su **business/ordini/nuovi**.

La moltiplicazione della flotta è ottenuta scalando le repliche del Deployment **drone-simulator**: poiché **DRONE.ID** è derivato dal **metadata.name** del Pod, ogni replica corrisponde a un drone digitale distinto. Lo scaling è invocabile anche dall'agente tramite il tool MCP **scale_drone_deployment** (patch su **deployment/scale**), con i guardrail descritti nel Capitolo 4.

2.3 Strategia di Osservabilità

L'osservabilità si fonda su tre pilastri:

1. **Dashboard web centralizzata** ospitata dal **central_server** con endpoint REST Flask (**/api/status**, **/api/orders**, **/api/drones**, **/api/completed**); il client esegue polling **fetch()** ogni 5 s.
2. **Log strutturati e audit trail**: **audit_actions.jsonl** (JSON Lines, traccia ogni azione di scrittura/rimediazione dell'IA: timestamp, tool, payload), **pending_approvals.jsonl** (ciclo di vita delle richieste HITL con giustificazione LLM) e log di runtime con tracking dei token consumati per ciascun ciclo.

3. **Serie temporali su InfluxDB:** bucket `iot_data` con i measurement `drone_telemetry` e `business_orders`; gli LLM analizzano time-window di 5/60 minuti tramite i tool MCP `get_drones_telemetry` e `get_pending_orders`.

Il registro `audit_actions.jsonl`, in particolare, traccia per ogni operazione di rimediazione il timestamp, il tipo di tool invocato e l'esatto payload, fornendo l'*auditability* richiesta dal dominio *high-stakes*. In parallelo, l'orchestratore logga per ciascuna iterazione il consumo di token restituito dai modelli, metrica cruciale per la verifica dei tetti di spesa discussi nel Capitolo 7.

3 Agentic AI e Design MCP

3.1 Il Limite dell'Automazione Deterministica

Tradizionalmente l'assegnazione degli ordini sarebbe gestita con un approccio deterministico basato su alberi decisionali (*scripting if-else*). Sebbene questo metodo sia efficiente per carichi di lavoro lineari, si rivela inadeguato nel dominio affrontato: ogni missione è influenzata dalla combinazione di priorità, peso del carico, batteria residua, tasso di usura del mezzo e distanza dall'HUB. Scrivere regole rigide per coprire ogni combinazione produrrebbe codice fragile e difficilmente manutenibile. Gli agenti superano questo limite con un ragionamento probabilistico che valuta dinamicamente i trade-off operativi e si adatta a contesti imprevedibili senza pre-programmazione esplicita.

3.2 Architettura MCP

Per garantire confini sicuri, il sistema adotta il *Model Context Protocol* (MCP) come strato di interposizione tra i loop LLM (in `logistic_ai_brain.py`) e l'infrastruttura. L'LLM non possiede credenziali dirette né permessi di root sui database: comunica via HTTP con il server MCP, che espone due categorie di tool:

- **Observer (read-only):** `get_drones_status` (API K8s), `get_drones_telemetry`, `get_pending_orders` (InfluxDB).
- **Remediation (write):** `send_mqtt_command` (assegnazione missioni) e `scale_drone_deployment` (auto-scaling Pod), entrambi soggetti alle policy del Capitolo 4.

3.3 Health Agent e Logistic Agent

HealthAgent. Supervisore della salute infrastrutturale: analizza il rapporto ordini pendenti/-capacità di flotta e decide lo scaling dei Pod per preservare gli SLO. Identifica anche guasti fisici: se un drone è in stato MAINTENANCE ma le coordinate divergono dall'HUB, deduce uno schianto e innesca le procedure di recupero.

LogisticAgent. Gestisce la coda ordini e l'assegnazione delle missioni. Incrocia gli ordini con lo stato dei droni IDLE, ponderando tre fattori: (i) peso del pacco (che riduce velocità e aumenta consumo), (ii) distanza euclidea (con calcolo preventivo dell'autonomia), (iii) batteria residua e usura. Ordini *high priority* con peso prossimo a 5.0 kg non vengono assegnati a droni con usura > 70% o batteria insufficiente. Selezionata la coppia ottimale, l'agente invoca `send_mqtt_command`.

3.4 Orchestrazione Dinamica

L'orchestratore (`logistic_ai_brain.py`) coordina gli agenti tramite due meccanismi chiave:

1. **Triage Manager e Context Injection:** l'orchestratore analizza lo snapshot del sistema, decide quale agente attivare e gli inietta nel prompt iniziale solo i dati rilevanti (es. solo i droni IDLE al `LogisticAgent`), portando l'LLM direttamente alla fase di invocazione dei tool.
2. **Esecuzione parallela:** se servono sia manutenzione/scaling sia assegnazione ordini, i due agenti vengono lanciati in parallelo su `threading.Thread` separati, dimezzando la latenza del ciclo decisionale.

4 Operational Safety e Agent Guardrails

L'introduzione di agenti LLM espone il sistema a rischi (allucinazioni, latenze, comportamenti non deterministici): il progetto adotta più livelli di guardrail per garantire azioni verificabili, confinate e sicure.

4.1 Least Privilege e Protezione Anti-Loop

L'IA agisce da *assistente operativo*, mai con controllo illimitato. Il server MCP è l'unico *chokepoint* verso l'infrastruttura e applica:

- **Accesso read-only alla telemetria:** solo `query_api` pre-compilate in Python, senza facoltà di comporre query *Flux/SQL* arbitrarie (rischio injection annullato).
- **Divieto di azioni distruttive:** nessun tool per DROP di bucket, DELETE di ordini storici o terminazione di processi core; il codice in `drone_mcp_layer.py` omette intenzionalmente queste implementazioni.

Per evitare loop di ragionamento infiniti, l'`HealthAgent` ha un contatore di iterazioni per turno (limite 3) e un controllo di formato sulla risposta: due output consecutivi non conformi terminano il processo. È inoltre presente un *kill-switch* contro lo scollegamento del backend: se le chiamate al server MCP falliscono oltre una soglia, l'attività degli agenti viene congelata e ripresa automaticamente al ritorno online del backend, evitando di consumare token su contesti vuoti.

4.2 Idempotenza delle Azioni

L'idempotenza è una proprietà di sicurezza fondamentale per evitare fallimenti catastrofici dovuti a loop o allucinazioni. Il meccanismo di scaling è l'esempio canonico: `scale_drone_deployment` invoca `patch_namespaced_deployment_scale` passando il *numero di repliche desiderate* (stato desiderato dichiarativo). Se l'LLM ripete il comando in loop, Kubernetes verifica che il Deployment ha già raggiunto la quota richiesta e ignora le chiamate successive. Una progettazione *incrementale* (aggiungere N droni per chiamata) avrebbe invece, in caso di allucinazione, saturato il cluster in pochi secondi, congestionato la rete MQTT e violato i tetti di spesa.

4.3 Human-In-The-Loop ed Escalation

La regola architetturale dei 6 droni. La costante `POLICY_LIMITS` in `drone_mcp_layer.py` fissa a 6 il numero massimo di droni scalabili in autonomia (`max_drones_auto`):

- ≤ 6 : l'`HealthAgent` invoca direttamente `scale_drone_deployment`.
- > 6 : il layer MCP blocca il comando e l'agente è obbligato a invocare `request_human_approval`.

Il portale di approvazione. Le richieste sono gestite da un servizio Flask indipendente (`human_approval_manager.py`, porta 5002) che espone una dashboard dedicata agli operatori. Quando l'agente richiede un'escalation, il server MCP persiste la richiesta in `pending_approvals.jsonl` e la dashboard renderizza una scheda con `request_id`, azione, payload JSON (es. `{"replicas": 8}`) e, soprattutto, la giustificazione in linguaggio naturale fornita dall'LLM (`reason`). L'operatore dispone di un diritto di veto assoluto: **Approve** convalida la transizione ed esegue istantaneamente il patch su Kubernetes con `force=True`, scavalcando i blocchi preventivi; **Deny**

contrassegna la richiesta come **rejected** e al ciclo successivo l'**HealthAgent** mantiene lo stato della flotta inalterato.

5 Reliability e Scalabilità

L'architettura sfrutta i comportamenti *out-of-the-box* delle librerie di base, arricchiti da pattern per confinare gli errori e rendere il sistema reattivo al traffico.

5.1 Resilienza di Rete

In fase di avvio i droni eseguono un `retry` custom con `time.sleep` di 5s qualora il broker non sia ancora schedulato. A regime, la resilienza è demandata alla libreria `paho-mqtt` e al suo `loop_start()`: in caso di caduta di Mosquitto il client avvia autonomamente *exponential backoff* per la riconnessione e ripristina la sottoscrizione ai topic `comandi/#` senza logica custom.

5.2 Graceful Degradation del Server Centrale

Per evitare che un downtime di InfluxDB blocchi la dashboard live, l'handler `on_message` di `central_server.py` adotta un pattern di *dual-write* con eccezione neutralizzata:

```
1 def on_message(client, userdata, msg):
2     state["drones"][drone_id] = parsing(msg.payload) # 1. in-memory
3     try:
4         write_api.write(bucket, org, record)           # 2. persistenza
5     except InfluxDBError:
6         log_warning("InfluxDB non raggiungibile.")    # degradazione
```

Listing 5.1: Pattern di Graceful Degradation.

Se InfluxDB è offline, vengono persi solo i log storici, mentre il tracciamento real-time e la dashboard restano operativi, isolando il guasto al solo storage layer.

5.3 Scalabilità Dinamica e Safety Limits

L'architettura supera lo scaling passivo basato su CPU/RAM delegando la valutazione agli agenti. Il tool MCP `scale_drone_deployment` invoca `patch_namespaced_deployment_scale` sul Deployment `drone-simulator`, variando in tempo reale le repliche in base alla coda. Lo scaling è subordinato al guardrail `POLICY_LIMITS`: oltre soglia l'azione è bloccata ed escalata al *Human Approval Manager* (Capitolo 4), evitando consumi cloud incontrollati per allucinazioni del modello.

```
1 if requested_replicas > POLICY_LIMITS:
2     trigger_human_escalation(requested_replicas) # escalation HITL
3 else:
4     k8s_client.patch_namespaced_deployment_scale( # esecuzione patch
5         name="drone-simulator", ...)
```

Listing 5.2: Guardrail di sicurezza applicato alle richieste di scaling dell'agente.

6 Chaos Engineering e Test di Fallimento

La suite `ops/chaos_test_suite.py` inietta guasti controllati sui componenti critici (database, broker, capacità di flotta) per misurare la reazione del sistema in termini di RTO e capacità di auto-rimediazione. L'obiettivo non è dimostrare l'assenza di guasti, ma misurare empiricamente la *reazione* dell'architettura discussa nel Capitolo 5.

6.1 Metodologia

Ogni esperimento segue il ciclo classico del chaos engineering:

1. **Ipotesi:** comportamento atteso e SLO da rispettare.
2. **Blast radius:** guasto confinato a un singolo Deployment in namespace `default`; il control-plane non viene mai toccato.
3. **Iniezione:** via API Kubernetes (`patch_namespaced_deployment_scale`, `delete_namespaced_pod`) o tramite generatore di carico (`client_simulator.py --stress`).
4. **Osservazione:** metriche su `chaos_test_results.log` (timestamp, RTO, repliche prima/dopo).
5. **Abort condition:** timeout massimo di 60 s per il ripristino, oltre il quale il test è considerato fallito.

6.2 Esperimenti e Risultati

(1) **Primary Database Outage.** Scaling di `influxdb` a 0 repliche, attesa 10s, ripristino a 1; cronometraggio del ritorno a `Ready`. *Risultato:* RTO = 12,05s. Il `central_server` ha continuato a servire la dashboard dallo stato in-memory; solo le scritture storiche sono andate perse, coerentemente col pattern di dual-write (§5.2).

(2) **MQTT Broker Crash.** Deletion del Pod `mosquitto` via `delete_namespaced_pod`, attesa del nuovo Pod `Running`. *Risultato:* downtime di scheduling K8s = 0,04s. La riconnessione dei droni è gestita nativamente da `paho-mqtt` senza logica custom; la telemetria *at-most-once* (2s) tollera le perdite.

(3) **Load Spike e Scaling Agentico.** Iniezione di 50 ordini con `client_simulator.py --stress` (durata ≈ 56 s), attesa ulteriore di 30 s per il ciclo decisionale e rilettura delle repliche. *Risultato:* repliche prima = 3, dopo = 6. L'orchestratore agentico ha rilevato lo spike, invocato il tool MCP di scaling e raddoppiato la flotta entro la finestra osservativa (≈ 86 s end-to-end), confermando l'efficacia dello scaling agentico per shock di carico controllati.

Esperimento	Metrica	Valore
InfluxDB outage	RTO	12,05 s
MQTT broker crash	Downtime K8s	0,04 s
Load spike (50 ord.)	Δ repliche	$3 \rightarrow 6$ (≈ 86 s)

Tabella 6.1: Run `chaos_test_suite.py` del 23/05/2026.

6.3 Lezioni Apprese

Gli esperimenti confermano la robustezza dei livelli infrastrutturali (Kubernetes + `paho-mqtt` + dual-write) e validano lo scaling agentico su orizzonti di ≈ 1 min. Restano due limiti: (i) la latenza del ciclo decisionale (≈ 30 s) è ancora elevata per spike sub-minuto, dove un HPA nativo offrirebbe un *fast-path* complementare; (ii) la suite non misura ancora la *loss rate* di telemetria durante l'outage di InfluxDB né la latenza end-to-end sotto stress. Tali estensioni sono discusse nel Capitolo 8.

7 Analisi Economica: SLO, Costi e ROA

7.1 SLO e Costo del Downtime

Sono stati definiti tre SLO coerenti col workload:

- **SLO 1 – Latenza assegnazione:** 90% degli ordini *high priority* assegnati entro 1 minuto (giustificato dal `time.sleep(30)` + latenza inferenza).
- **SLO 2 – Disponibilità data layer:** 99,9% di uptime per MQTT e InfluxDB; una perdita prolungata renderebbe cieco l'HealthAgent.
- **SLO 3 – Affidabilità agentica:** 99% di tool call valide senza innesco di `MAX_AGENT_ITERATIONS`.

Nel dominio della logistica critica, un'ora di downtime è stimata a circa **15.000 €/ora** (media dei guadagni orari + penali contrattuali per SLA violati).

7.2 Return on Agent (ROA): OPEX e CAPEX

L'orchestrazione autonoma introduce un bilanciamento critico tra i costi operativi delle API (OPEX) e il risparmio di capitale hardware (CAPEX) ottenuto via logiche predittive.

Ottimizzazione OPEX. Il sistema limita l'impatto economico delle chiamate LLM attraverso tre strategie integrate:

- *Triage preventivo:* il `triage_manager` analizza lo snapshot del sistema prima di invocare i modelli, attivando `LogisticAgent` ed `HealthAgent` solo se sussistono reali condizioni di sbilanciamento (es. ordini pendenti accoppiati a droni idle, o necessità di scaling/manutenzione), azzerando i costi dei cicli no-op ridondanti.
- *Ingegneria dei prompt:* i prompt di sistema impongono restrizioni sintattiche che vietano la generazione di testo libero, preamboli o spiegazioni discorsive; l'output è confinato alle sole chiamate di funzione (*tool calling*), minimizzando i token di completamento.
- *Monitoraggio consumi:* auditing sistematico dei token (prompt, completamento, totale) al termine di ogni esecuzione, per rendicontazione finanziaria precisa del processo decisionale.

Preservazione CAPEX. Il ritorno sull'investimento si concretizza nella mitigazione dei rischi d'impresa e nell'ottimizzazione infrastrutturale:

- *Manutenzione preventiva dinamica:* il `LogisticAgent` include nei prompt la regola di evitare carichi $> 3\text{ kg}$ su droni con usura $> 70\%$ (*soft constraint* prompt-driven), riducendo il rischio di esaurimento batteria e perdita del vettore.
- *Elasticità infrastrutturale cloud:* l'HealthAgent modula dinamicamente le repliche del Deployment in base al carico effettivo, riducendo i costi di over-provisioning.
- *Operational Safety Cap:* limite massimo di 6 droni in autonomia (`POLICY_LIMITS`); qualsiasi espansione ulteriore richiede `request_human_approval`, ponendo un *budget cap* rigido contro loop di scaling incontrollati.
- *Massimizzazione del throughput:* la coda ordini è ordinata per priorità, con l'agente che alloca la capacità residua privilegiando i task *high priority*.

Proiezioni Economiche

Ipotizzando 2.880 cicli/giorno (polling ogni 30 s):

Voce	Gemini 2.5 Pro	Mistral Small
Infrastruttura cloud (1 control-plane + 2 worker)	150,00 \$/mese	150,00 \$/mese
Token LLM	~500,00 \$/mese	~10,00 \$/mese
OPEX totale mensile	~650,00 \$/mese	~160,00 \$/mese

Tabella 7.1: Confronto OPEX al variare del modello LLM.

La forbice di costo (delta ~490 \$/mese) consente di modulare la scelta del modello: *Mistral Small* per scenari ordinari di instradamento; *Gemini 2.5 Pro* riservato a finestre critiche con anomalie complesse o conflitti di policy.

7.3 Trade-off Costo/Affidabilità e Automazione/Sicurezza

Costo vs. affidabilità. Un sistema deterministico presenta OPEX minimi (esecuzione di script locali) ma soffre di scarsa resilienza: a fronte di eccezioni non gestite o anomalie di rete subisce arresti critici, e il ripristino richiede l'intervento manuale, incrementando i costi di reperibilità oraria e introducendo latenze di risoluzione. Il sistema multi-agente paga un costo variabile in token (fino a 500 \$/mese con *Gemini 2.5 Pro*) in cambio della flessibilità semantica del modello, che contestualizza l'errore, ragiona sulla deviazione e devia su flussi alternativi senza richiedere intervento umano.

Automazione vs. sicurezza. Il deterministico garantisce l'osservanza rigorosa delle policy ma è rigido davanti ad anomalie inattese (es. sovrapposizione di backlog di ordini e fermo contemporaneo di più vettori), provocando blocchi fino all'intervento manuale. Il multi-agente massimizza l'efficienza decisionale abilitando valutazioni di trade-off complessi tra metriche di business e vincoli fisici; l'elasticità decisionale introduce però rischi (allucinazioni nelle tool call, derive logiche sullo scaling) mitigati dai guardrail del Capitolo 4.

8 Conclusioni e Sviluppi Futuri

Il progetto ha dimostrato la fattibilità di un'architettura logistica resiliente in cui l'automazione è affidata a un layer *Agentic AI* mediato da MCP, con un orchestratore che funge da *triage manager* per **HealthAgent** (piano infrastrutturale) e **LogisticAgent** (piano di business). La separazione tra LLM, layer MCP e infrastruttura Kubernetes ha permesso di soddisfare contemporaneamente osservabilità, sicurezza e scalabilità.

Risultati principali. (i) *Reliability infrastrutturale*: graceful degradation, exponential backoff e rischedulazione K8s hanno garantito RTO contenuto (12 s su InfluxDB) e rischedulazione immediata del broker MQTT senza interruzione del monitoraggio. (ii) *Safety agentica: Least Privilege*, contatori di iterazione, kill-switch backend e HITL hanno confinato l'autonomia dell'LLM; la soglia dei 6 droni mantiene il controllo umano sulle azioni ad alto impatto. (iii) *Sostenibilità economica*: OPEX prevedibile in intervallo 160–650 \$/mese, con scelta consapevole tra capacità di ragionamento e costo per ciclo.

Limitazioni. La latenza del ciclo agentico (`time.sleep(30)` + inferenza) non è adatta a load spike < 60 s; la regola peso/usura è oggi solo prompt-driven; la suite di chaos non misura loss rate di telemetria né latenza end-to-end sotto stress; i valori economici sono basati su ipotesi di carico costante.

Sviluppi futuri. (i) Affiancare un HPA Kubernetes nativo come *fast-path* per i picchi a breve orizzonte, lasciando all'agente le decisioni strategiche. (ii) Promuovere la regola peso/usura da *soft policy* a vincolo deterministico nel layer MCP. (iii) Estendere la suite di chaos engineering con latency injection, misurazione della loss rate di InfluxDB e validazione del kill-switch in outage prolungati. (iv) Implementare un routing dinamico tra modelli LLM (Mistral Small per ordinario, Gemini 2.5 Pro per anomalie complesse) per massimizzare il ROA. (v) Validare il digital twin su dispositivi fisici o simulatori ad alta fedeltà.

In sintesi, il progetto mostra come un layer agentico mediato da MCP, accompagnato da guardrail di sicurezza e da una rigorosa strategia di osservabilità, possa elevare il livello di automazione di un'infrastruttura logistica critica senza sacrificare la prevedibilità operativa e il controllo umano sulle decisioni ad alto impatto.